

Healthcare Analysis

Gao

2024-02-28

```
library(tidymodels)
```

```
## -- Attaching packages ----- tidymodels 1.1.1 --
```

```
## v broom      1.0.5    v recipes      1.0.9
## v dials      1.2.0    v rsample     1.2.0
## v dplyr      1.1.4    v tibble     3.2.1
## v ggplot2    3.4.4    v tidyr      1.3.1
## v infer      1.0.6    v tune       1.1.2
## v modeldata  1.3.0    v workflows  1.1.3
## v parsnip    1.1.1    v workflowsets 1.0.1
## v purrr      1.0.2    v yardstick  1.3.0
```

```
## -- Conflicts ----- tidymodels_conflicts() --
```

```
## x purrr::discard() masks scales::discard()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## x recipes::step() masks stats::step()
## * Use tidymodels_prefer() to resolve common conflicts.
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v forcats  1.0.0    v readr      2.1.5
## v lubridate 1.9.3    v stringr    1.5.1
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x readr::col_factor() masks scales::col_factor()
## x purrr::discard()    masks scales::discard()
## x dplyr::filter()     masks stats::filter()
## x stringr::fixed()    masks recipes::fixed()
## x dplyr::lag()        masks stats::lag()
## x readr::spec()       masks yardstick::spec()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(rpart.plot)
```

```
## Loading required package: rpart
##
```

```
## Attaching package: 'rpart'
##
## The following object is masked from 'package:dials':
##
##      prune
```

```
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following objects are masked from 'package:yardstick':
##
##      precision, recall, sensitivity, specificity
##
## The following object is masked from 'package:purrr':
##
##      lift
```

```
library(dplyr)
```

1-5.

```
Watson <- read_csv("Watson_Healthcare_Modified.csv")
```

```
## Rows: 1676 Columns: 35
## -- Column specification -----
## Delimiter: ","
## chr (9): Attrition, BusinessTravel, Department, EducationField, Gender, Job...
## dbl (26): EmployeeID, Age, DailyRate, DistanceFromHome, Education, EmployeeC...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

6. Convert the Attrition column from the character type to the factor type. This is necessary because the dependent variable in a classification model must be of the factor type.

```
Watson$Attrition <- as.factor(Watson$Attrition)
```

7. Drop all columns of the character type that only have a single unique value. This is necessary because having an independent variable that only has a single unique values causes an error if you attempt to tune a classification model.

I chose these three variables by looking at the tibble in the environment pane

```
Watson <- select(Watson, -c(EmployeeCount, Over18, StandardHours))
```

8. Split the data into testing and training sets

I used a 75-25 split

```
Watson_split <- initial_split(Watson, prop = 0.75)
train <- training(Watson_split)
test <- testing(Watson_split)
```

9. Create a decision tree model with a depth of 30 and a minimum of 5 data points per node

```
dtree_mod <- decision_tree(mode = "classification", min_n = 5, tree_depth = 30)
```

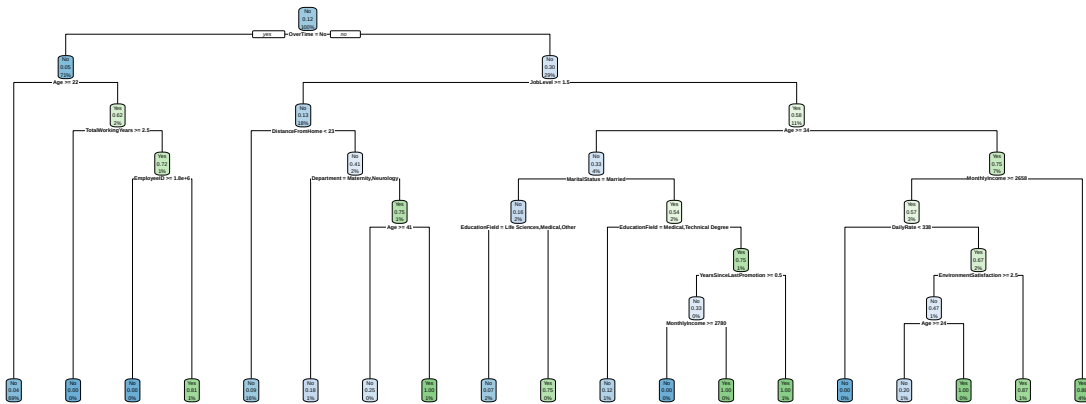
10. Fit the model to predict the Attrition variable given all of the remaining variables in the data set.

```
dtree_mod_fit <- fit(object = dtree_mod, formula = Attrition ~ ., data = train)
```

11. Plot the model.

```
dtree_mod_fit %>% extract_fit_engine() %>% rpart.plot()
```

```
## Warning: Cannot retrieve the data used to build the model (so cannot determine roundint and is.binary)
## To silence this warning:
##     Call rpart.plot with roundint=FALSE,
##     or rebuild the rpart model with model=TRUE.
```



12. Create the predictions for the model.

```
predictions <- predict(dtree_mod_fit, new_data = test)
```

```
predictions
```

```
## # A tibble: 419 x 1
##   .pred_class
##   <fct>
## 1 No
## 2 No
## 3 Yes
## 4 No
## 5 No
## 6 Yes
## 7 No
## 8 No
## 9 No
## 10 No
## # i 409 more rows
```

13. View the confusion matrix for the model as well as the accuracy and kappa metrics.

```
confusionMatrix(data = predictions$.pred_class, reference = test$Attrition)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  No Yes
##           No 361 26
##           Yes 13 19
##
##           Accuracy : 0.9069
##           95% CI : (0.875, 0.933)
##           No Information Rate : 0.8926
##           P-Value [Acc > NIR] : 0.19408
##
##           Kappa : 0.4439
##
## Mcnemar's Test P-Value : 0.05466
##
##           Sensitivity : 0.9652
##           Specificity : 0.4222
##           Pos Pred Value : 0.9328
##           Neg Pred Value : 0.5937
##           Prevalence : 0.8926
##           Detection Rate : 0.8616
##           Detection Prevalence : 0.9236
##           Balanced Accuracy : 0.6937
##
##           'Positive' Class : No
##
```

14. Check the variable importance of the model. Make a note of any variables that have an importance of 0.

As we can see, Education, PerformanceRating, RelationshipSatisfaction and TrainingTimesLastYear all have an importance of 0

```
dtree_mod_fit %>% extract_fit_engine() %>% varImp()
```

```
##              Overall
## Age          98.732428
## DailyRate    8.076686
## Department   7.537987
## DistanceFromHome 8.205691
## Education    4.033333
## EducationField 19.546526
## EmployeeID   10.312083
## EnvironmentSatisfaction 3.788889
## Gender       3.849206
## JobInvolvement 12.438800
## JobLevel     57.199487
## JobRole      11.533897
## MaritalStatus 22.287750
## MonthlyIncome 91.204319
## OverTime     32.081206
## PercentSalaryHike 8.389287
## Shift        18.836247
## TotalWorkingYears 81.167716
## TrainingTimesLastYear 3.103030
## WorkLifeBalance 1.767677
## YearsAtCompany 31.833362
## YearsInCurrentRole 3.103030
## YearsSinceLastPromotion 6.738182
## YearsWithCurrManager 9.055302
## BusinessTravel 0.000000
## HourlyRate    0.000000
## JobSatisfaction 0.000000
## MonthlyRate   0.000000
## NumCompaniesWorked 0.000000
## PerformanceRating 0.000000
## RelationshipSatisfaction 0.000000
```

15. Create six cross validation folds for the training data

```
folds <- vfold_cv(train, v = 6)
```

16. Create an automatic grid of hyperparameters for the model

```
tree_grid_auto <- grid_regular(tree_depth(), min_n(), cost_complexity())
```

```
tree_grid_auto
```

```
## # A tibble: 27 x 3
##   tree_depth min_n cost_complexity
##   <int> <int>      <dbl>
## 1         1     2  0.0000000001
## 2         8     2  0.0000000001
## 3        15     2  0.0000000001
## 4         1    21  0.0000000001
## 5         8    21  0.0000000001
## 6        15    21  0.0000000001
## 7         1    40  0.0000000001
## 8         8    40  0.0000000001
## 9        15    40  0.0000000001
## 10        1     2  0.00000316
## # i 17 more rows
```

```
print(tree_grid_auto, n = 27)
```

```
## # A tibble: 27 x 3
##   tree_depth min_n cost_complexity
##   <int> <int>      <dbl>
## 1         1     2  0.0000000001
## 2         8     2  0.0000000001
## 3        15     2  0.0000000001
## 4         1    21  0.0000000001
## 5         8    21  0.0000000001
## 6        15    21  0.0000000001
## 7         1    40  0.0000000001
## 8         8    40  0.0000000001
## 9        15    40  0.0000000001
## 10        1     2  0.00000316
## 11         8     2  0.00000316
## 12        15     2  0.00000316
## 13         1    21  0.00000316
## 14         8    21  0.00000316
## 15        15    21  0.00000316
## 16         1    40  0.00000316
## 17         8    40  0.00000316
## 18        15    40  0.00000316
## 19         1     2  0.1
## 20         8     2  0.1
## 21        15     2  0.1
## 22         1    21  0.1
## 23         8    21  0.1
## 24        15    21  0.1
## 25         1    40  0.1
## 26         8    40  0.1
## 27        15    40  0.1
```

17. Use the grid and the folds to tune the model

```
dtree_mod_tune <- decision_tree(mode = "classification", tree_depth = tune(), min_n = tune(), cost_complexity = tune())
tuning_data <- tune_grid(dtree_mod_tune, preprocessor = Attrition ~., resamples = folds, grid = tree_grid())
print(tuning_data %>% collect_metrics, n = 54)
```

```
## # A tibble: 54 x 9
##   cost_complexity tree_depth min_n .metric .estimator mean n std_err
##   <dbl> <int> <int> <chr> <chr> <dbl> <int> <dbl>
## 1 0.0000000001 1 2 accuracy binary 0.868 6 0.00686
## 2 0.0000000001 1 2 roc_auc binary 0.514 6 0.0108
## 3 0.0000000001 8 2 accuracy binary 0.881 6 0.00411
## 4 0.0000000001 8 2 roc_auc binary 0.696 6 0.0548
## 5 0.0000000001 15 2 accuracy binary 0.872 6 0.00587
## 6 0.0000000001 15 2 roc_auc binary 0.712 6 0.0259
## 7 0.0000000001 1 21 accuracy binary 0.868 6 0.00686
## 8 0.0000000001 1 21 roc_auc binary 0.514 6 0.0108
## 9 0.0000000001 8 21 accuracy binary 0.889 6 0.00465
## 10 0.0000000001 8 21 roc_auc binary 0.803 6 0.00927
## 11 0.0000000001 15 21 accuracy binary 0.889 6 0.00465
## 12 0.0000000001 15 21 roc_auc binary 0.803 6 0.00927
## 13 0.0000000001 1 40 accuracy binary 0.868 6 0.00686
## 14 0.0000000001 1 40 roc_auc binary 0.514 6 0.0108
## 15 0.0000000001 8 40 accuracy binary 0.900 6 0.00406
## 16 0.0000000001 8 40 roc_auc binary 0.807 6 0.0108
## 17 0.0000000001 15 40 accuracy binary 0.900 6 0.00406
## 18 0.0000000001 15 40 roc_auc binary 0.807 6 0.0108
## 19 0.00000316 1 2 accuracy binary 0.868 6 0.00686
## 20 0.00000316 1 2 roc_auc binary 0.514 6 0.0108
## 21 0.00000316 8 2 accuracy binary 0.881 6 0.00411
## 22 0.00000316 8 2 roc_auc binary 0.696 6 0.0548
## 23 0.00000316 15 2 accuracy binary 0.872 6 0.00587
## 24 0.00000316 15 2 roc_auc binary 0.712 6 0.0259
## 25 0.00000316 1 21 accuracy binary 0.868 6 0.00686
## 26 0.00000316 1 21 roc_auc binary 0.514 6 0.0108
## 27 0.00000316 8 21 accuracy binary 0.889 6 0.00465
## 28 0.00000316 8 21 roc_auc binary 0.803 6 0.00927
## 29 0.00000316 15 21 accuracy binary 0.889 6 0.00465
## 30 0.00000316 15 21 roc_auc binary 0.803 6 0.00927
## 31 0.00000316 1 40 accuracy binary 0.868 6 0.00686
## 32 0.00000316 1 40 roc_auc binary 0.514 6 0.0108
## 33 0.00000316 8 40 accuracy binary 0.900 6 0.00406
## 34 0.00000316 8 40 roc_auc binary 0.807 6 0.0108
## 35 0.00000316 15 40 accuracy binary 0.900 6 0.00406
## 36 0.00000316 15 40 roc_auc binary 0.807 6 0.0108
## 37 0.1 1 2 accuracy binary 0.877 6 0.00640
## 38 0.1 1 2 roc_auc binary 0.5 6 0
## 39 0.1 8 2 accuracy binary 0.879 6 0.00650
## 40 0.1 8 2 roc_auc binary 0.537 6 0.0371
## 41 0.1 15 2 accuracy binary 0.879 6 0.00650
## 42 0.1 15 2 roc_auc binary 0.537 6 0.0371
## 43 0.1 1 21 accuracy binary 0.877 6 0.00640
```

```
## 44    0.1          1    21 roc_auc binary    0.5      6 0
## 45    0.1          8    21 accuracy binary  0.879    6 0.00650
## 46    0.1          8    21 roc_auc binary    0.537    6 0.0371
## 47    0.1         15    21 accuracy binary  0.879    6 0.00650
## 48    0.1         15    21 roc_auc binary    0.537    6 0.0371
## 49    0.1          1    40 accuracy binary  0.877    6 0.00640
## 50    0.1          1    40 roc_auc binary    0.5      6 0
## 51    0.1          8    40 accuracy binary  0.879    6 0.00650
## 52    0.1          8    40 roc_auc binary    0.537    6 0.0371
## 53    0.1         15    40 accuracy binary  0.879    6 0.00650
## 54    0.1         15    40 roc_auc binary    0.537    6 0.0371
## # i 1 more variable: .config <chr>
```

18. Display the best hyperparameter values for the model

The best hyperparameter values are `min_n = 40`, and `tree_depth = 8`

```
tuning_data %>% select_best(metric = "accuracy")
```

```
## # A tibble: 1 x 4
##   cost_complexity tree_depth min_n .config
##         <dbl>         <int> <int> <chr>
## 1    0.0000000001             8    40 Preprocessor1_Model108
```

```
tuning_data %>% show_best(metric = "accuracy")
```

```
## # A tibble: 5 x 9
##   cost_complexity tree_depth min_n .metric .estimator mean      n std_err
##         <dbl>         <int> <int> <chr>    <chr>    <dbl> <int>    <dbl>
## 1    0.0000000001             8    40 accuracy binary    0.900     6 0.00406
## 2    0.0000000001            15    40 accuracy binary    0.900     6 0.00406
## 3    0.00000316              8    40 accuracy binary    0.900     6 0.00406
## 4    0.00000316            15    40 accuracy binary    0.900     6 0.00406
## 5    0.0000000001             8    21 accuracy binary    0.889     6 0.00465
## # i 1 more variable: .config <chr>
```

19. Create the final model that uses the best hyperparameter values.

```
dtree_mod_final <- decision_tree(mode = "classification", min_n = 40, tree_depth = 8)
```

20. Fit the final model without using any variables that have an importance of 0.

```
dtree_mod_fit_final <- fit(object = dtree_mod, formula = Attrition ~ . - Education - PerformanceRating)
```

21. Use the final model to create predictions.

```
predictions_final <- predict(dtree_mod_fit_final, new_data = test)

predictions_final
```

```
## # A tibble: 419 x 1
##   .pred_class
##   <fct>
## 1 No
## 2 No
## 3 Yes
## 4 No
## 5 No
## 6 Yes
## 7 No
## 8 No
## 9 No
## 10 No
## # i 409 more rows
```

22. View the confusion matrix for the final model

```
confusionMatrix(data = predictions_final$.pred_class, reference = test$Attrition)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  No  Yes
##           No 356  24
##           Yes  18  21
##
##               Accuracy : 0.8998
##               95% CI : (0.8669, 0.9268)
##           No Information Rate : 0.8926
##           P-Value [Acc > NIR] : 0.3532
##
##               Kappa : 0.4446
##
##  Mcnemar's Test P-Value : 0.4404
##
##           Sensitivity : 0.9519
##           Specificity : 0.4667
##           Pos Pred Value : 0.9368
##           Neg Pred Value : 0.5385
##           Prevalence : 0.8926
##           Detection Rate : 0.8496
##           Detection Prevalence : 0.9069
##           Balanced Accuracy : 0.7093
##
##           'Positive' Class : No
##
```